

STORE MANAGEMENT API

Technical Documentation

Version: 1.0.0

Status: Final Release

Project Type: REST API

Author: Leon Santiago Gomes (leonndas)

Technologies: Java • Spring Boot • Spring Data JPA • Hibernate • MySQL • Maven • Git
• GitHub • Postman

Repository

<https://github.com/leonndas/store-management-api>

Last Updated

August 2026

Document Control

Item	Information
Document Title	Store Management API – Technical Documentation
Version	1.0.0
Status	Final
Author	Leon Santiago Gomes
Language	English
Document Type	Software Technical Documentation
Repository	Store Management API
Target Audience	Recruiters, Software Developers and Technical Reviewers

Revision History

Version	Date	Author	Description
1.0.0	August 2026	Leon Santiago Gomes	Initial release

Table of Contents

I.	Executive Summary
II.	Project Overview
III.	Business Context
IV.	Project Objectives
V.	Functional Requirements
VI.	Non-Functional Requirements
VII.	System Architecture
VII.I	System Architecture Diagram
VIII.	MVC Architecture
VIII.I	Model Layer
VIII.I.I	Controller Layer
VIII.I.II	Service Layer
VIII.I.III	Repository Layer
IX.	Technology Stack
IX.I	Why These Technologies?
IX.I.I	Java
IX.I.II	Spring Boot
IX.I.III	Spring Data JPA

IX.I.IV	Hibernate
IX.I.V	MySQL
IX.I.VI	Maven
IX.I.VII	Git & GitHub
IX.I.VIII	Postman

X. Project Structure

X.I Package Responsibilities

X.I.I	Controller
X.I.II	Model
X.I.III	Repository
X.I.IV	Service
X.I.V	Resources
X.I.VI	Pom.xml

XI. Database Design

XI.I Entity Relationships

XII. REST API Overview

XII.I Base URL

XIII. Customer API

XIV. Product API

XV. Order API

XVI. Postman Test Evidence

XVII. Installation Guide

XVII.I Clone Repository

XVII.I Navigate to Project

XVII.I Configure Database

XVII.I Run Application

XVIII. Conclusion

XIX. References

XX. Appendix

1. Executive Summary

The Store Management API is a RESTful backend application developed to simulate the core operations of a retail management system. The project focuses on providing a clean, maintainable and scalable architecture for managing customers, products and purchase orders through HTTP endpoints.

The application was designed following modern backend development practices using the Spring ecosystem, allowing data persistence through a relational database while exposing RESTful services that communicate using JSON.

Beyond demonstrating CRUD operations, this project also emphasizes software organization, separation of responsibilities, object-relational mapping, and API design principles commonly adopted in enterprise applications.

The main goal of this project is to demonstrate practical experience with backend development using Java and Spring Boot while following industry standards regarding project organization, documentation and code structure.

2. Project Overview

The Store Management API was developed as a backend solution responsible for managing customer registrations, product information and purchase orders.

Instead of interacting directly with the database, external applications communicate with the API through REST endpoints, making the system modular and independent from any frontend technology.

The project follows the MVC (Model–View–Controller) architecture provided by Spring Boot, ensuring a clear separation between business logic, data persistence and request handling.

All data exchanged between client applications and the server uses the JSON format, while information is permanently stored in a MySQL relational database using Spring Data JPA and Hibernate.

3. Business Context

Retail businesses require reliable systems to organize daily operations and maintain accurate records of customers, products and sales.

The Store Management API was designed to provide these core functionalities through a lightweight backend capable of supporting future integrations with web applications, mobile applications or third-party systems.

The application centralizes business information, reducing manual processes and providing a standardized interface for accessing and manipulating data.

Although intentionally simple, the architecture was designed considering software engineering principles that allow future scalability and feature expansion.

4. Project Objectives

The primary objective of this project is to provide a RESTful backend capable of managing the basic operations of a store management system.

The specific objectives include:

- Develop a REST API using Spring Boot.
 - Implement CRUD operations for Customers.
 - Implement CRUD operations for Products.
 - Implement CRUD operations for Orders.
 - Persist application data using MySQL.
 - Apply the MVC architectural pattern.
 - Implement object-relational mapping through Spring Data JPA.
 - Expose standardized JSON endpoints.
 - Validate the API using Postman.
 - Maintain a clean and organized project structure.
-

5. Functional Requirements

The system shall allow users to register new customers.

The system shall allow users to retrieve customer information by identifier.

The system shall allow users to list all registered customers.

The system shall allow users to delete existing customers.

The system shall allow users to register new products.

The system shall allow users to retrieve products by identifier.

The system shall allow users to list available products.

The system shall allow users to delete products.

The system shall allow users to create purchase orders.

The system shall associate each order with exactly one customer.

The system shall associate each order with exactly one product.

The system shall store the quantity purchased.

The system shall return responses in JSON format.

6. Non-Functional Requirements

The application shall be implemented using Java and Spring Boot.

The application shall use Spring Data JPA for persistence.

The application shall use Hibernate as the ORM provider.

The application shall persist data in a MySQL relational database.

The API shall follow REST architectural principles.

The source code shall follow the MVC architecture.

The application shall support JSON request and response bodies.

The project shall be managed using Maven.

The source code shall be version-controlled using Git and hosted on GitHub.

The API shall be tested using Postman.

7. System Architecture

The Store Management API follows a layered architecture based on the MVC (Model–View–Controller) design pattern. This approach separates responsibilities between request handling, business logic, data persistence and database communication, improving maintainability, readability and scalability.

Client applications communicate with the REST API through HTTP requests. The Controller layer receives incoming requests, delegates business operations to the Service layer and returns responses in JSON format.

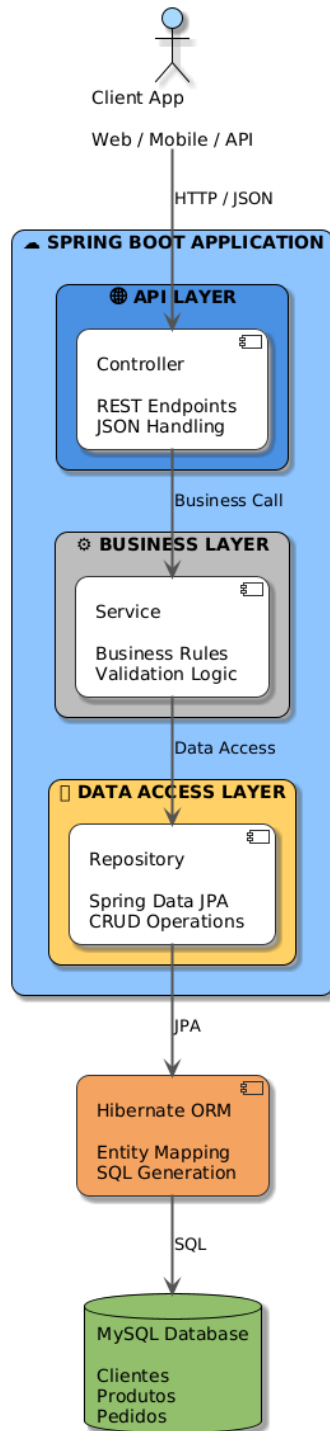
The Service layer contains the application's business rules and orchestrates interactions between controllers and repositories.

The Repository layer abstracts database operations by using Spring Data JPA, allowing CRUD operations without manually writing SQL queries.

Finally, Hibernate translates Java objects into relational database records stored in MySQL.

7.1 System Architecture Diagram

Baozi Store API System Architecture



8. MVC Architecture

The application adopts the Model–View–Controller architectural pattern provided by Spring Boot. Each layer has a specific responsibility, ensuring low coupling and high maintainability.

8.1 Model Layer

The Model layer represents the application's business entities. The following entities were implemented:

- Customer
- Product
- Order

These classes are mapped to relational database tables using JPA annotations.

8.1.1 Controller Layer

The Controller layer exposes REST endpoints responsible for receiving HTTP requests from client applications. Each controller processes incoming requests and delegates business operations to the corresponding service.

Implemented controllers:

- CustomerController
- ProductController
- OrderController

8.1.2 Service Layer

The Service layer contains the application's business logic. Responsibilities include:

- validating requests
- coordinating database operations
- communicating with repositories
- applying business rules

Separating business logic from controllers improves readability and facilitates future maintenance.

8.1.3 Repository Layer

Repositories are responsible for database communication. Spring Data JPA automatically provides CRUD implementations through the `JpaRepository` interface. Implemented repositories:

- `CustomerRepository`
- `ProductRepository`
- `OrderRepository`

This approach significantly reduces boilerplate code while maintaining excellent readability.

9. Technology Stack

The project was developed using modern backend technologies widely adopted by enterprise software development.

Technology	Purpose
Java 21	Main programming language
Spring Boot	REST API Framework
Spring Data JPA	Data persistence
Hibernate	Object Relational Mapping (ORM)
MySQL	Relational database
Maven	Dependency management
Git	Version control
GitHub	Source code hosting
Postman	API testing

9.1 Why These Technologies?

9.1.1 Java

Java provides excellent portability, object-oriented programming features and a mature ecosystem for enterprise applications.

9.1.2 Spring Boot

Spring Boot simplifies application configuration while providing production-ready features and dependency injection.

9.1.3 Spring Data JPA

Spring Data JPA minimizes repetitive database code by automatically implementing repository methods.

9.1.4 Hibernate

Hibernate manages object-relational mapping, converting Java objects into relational database records.

9.1.5 MySQL

MySQL offers reliable relational data storage with excellent performance and wide industry adoption.

9.1.6 Maven

Maven automates dependency management and project builds.

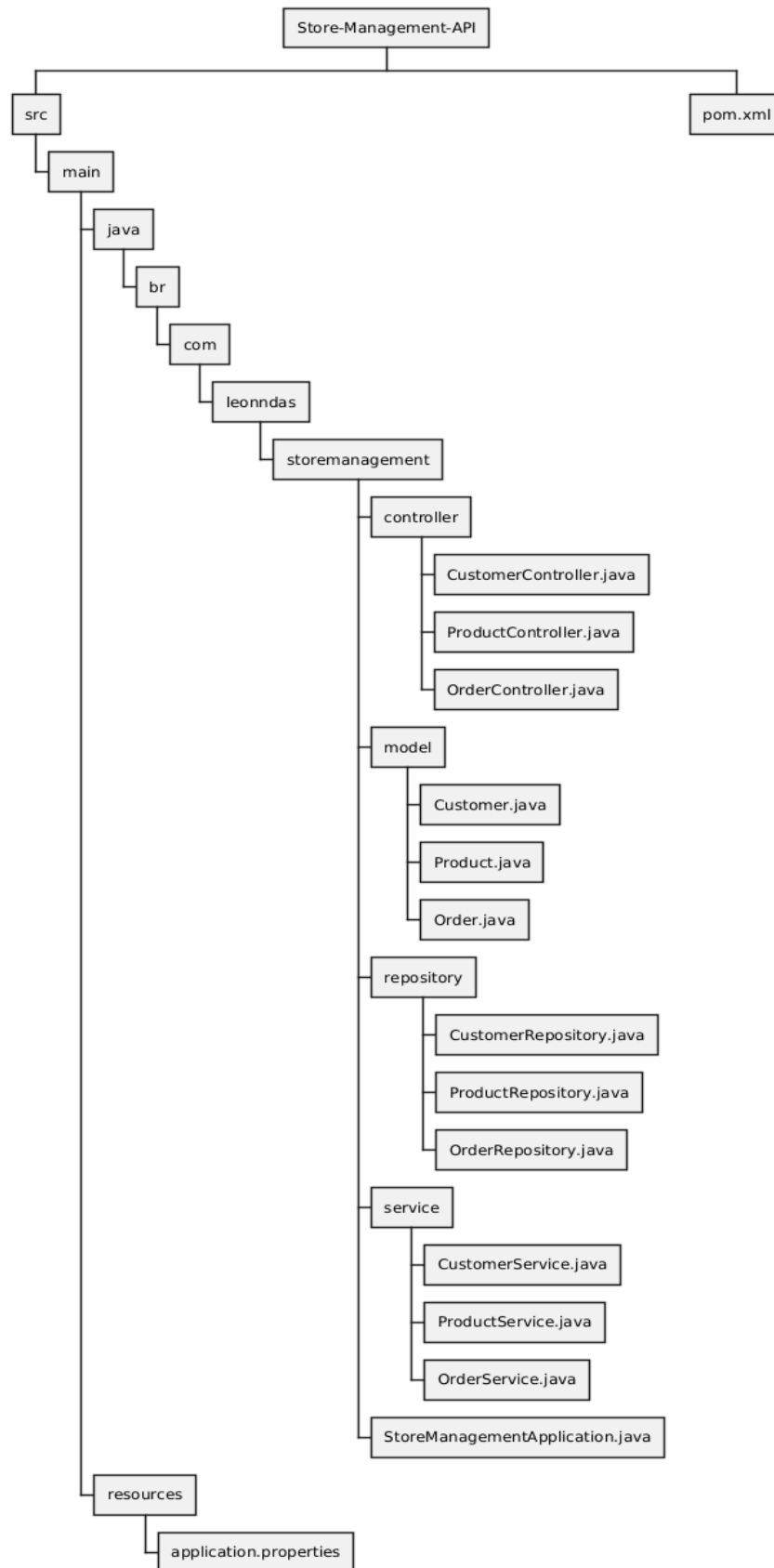
9.1.7 Git & GitHub

Git provides version control while GitHub enables collaboration and project hosting.

9.1.8 Postman

Postman simplifies API testing by allowing developers to create, execute and organize HTTP requests.

10. Project Structure



10.1 Package Responsibilities

The project follows the standard Maven directory structure recommended for Spring Boot applications.

10.1.1 Controller

Responsible for receiving HTTP requests and returning HTTP responses.

10.1.2 Model

Contains the application's entities mapped to database tables.

10.1.3 Repository

Provides CRUD operations through Spring Data JPA.

10.1.4 Service

Contains business logic and communication between controllers and repositories.

10.1.5 Resources

Stores configuration files such as application.properties.

10.1.6 Pom.xml

Manages project dependencies and build configuration.



11. Database Design

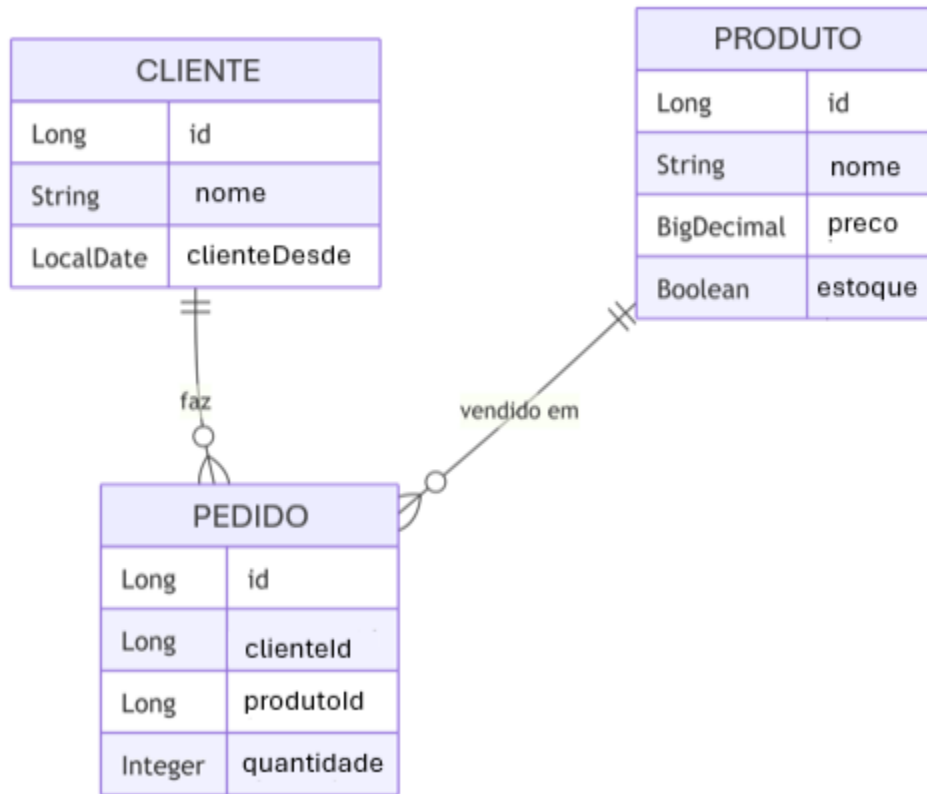
The application uses a relational database developed in MySQL. Three entities compose the system:

- Customer
- Product
- Order

Each order references exactly one customer and one product through foreign keys.

This relational model guarantees data consistency and simplifies future expansion of the application.

11.1 Entity Relationships



12. REST API Overview

The Store Management API follows the REST architectural style, exposing HTTP endpoints responsible for managing customers, products and orders.

All communication between client applications and the API is performed using the JSON format.

The application implements standard HTTP methods for resource manipulation while maintaining a consistent URI structure.

12.1 Base URL

<http://localhost:8080>

13. Customer API

Operation	Method	Endpoint	Request	Response
Create Customer	POST	/customers	<pre>{ "name": "Leon Santiago", "customerSince": "2026-08-06" }</pre>	<pre>{ "id": 1, "name": "Leon Santiago", "customerSince": "2026-08-06" }</pre>
Get All Customers	GET	/customers	—	<pre>[{ "id": 1, "name": "Leon Santiago", "customerSince": "2026-08-06" }]</pre>
Get Customer By ID	GET	/customers/{id}	—	<pre>{ "id": 1, "name": "Leon Santiago", "customerSince": "2026-08-06" } </pre>
Delete	DELETE	/customers/{id}	—	—

14. Product API

Operation	Method	Endpoint	Request	Response
Create Product	POST	/products	{ "name": "Baozi Pork", "price": 12.50, "stock": true }	{ "id": 1, "name": "Baozi Pork", "price": 12.50, "stock": true }
Get All Products	GET	/products	—	[[{ "id": 1, "name": "Baozi Pork", "price": 12.50, "stock": true }]]
Get Product By ID	GET	/products/{id}	—	{ "id": 1, "name": "Baozi Pork", "price": 12.50, "stock": true }
Delete Product	DELETE	/products/{id}	—	—

15. Order API

Operation	Method	Endpoint	Request	Response
Create Order	POST	/orders	<pre>{ "customer": {"id": 1}, "product": { "id": 1}, "quantity": 3 }</pre>	<pre>{ "id": 1, "customer": {"id": 1}, "product": { "id": 1 }, "quantity": 3 }</pre>
Get All Orders	GET	/orders	-	<pre>[{ "id": 1, "customer": {"id": 1}, "product": {"id": 1}, "quantity": 3 }]</pre>
Get Order By ID	GET	/orders/{id}	-	<pre>{ "id": 1, "customer": {"id": 1}, "product": {"id": 1}, "quantity": 3 }</pre>
Delete Order	DELETE	/orders/{id}	-	-

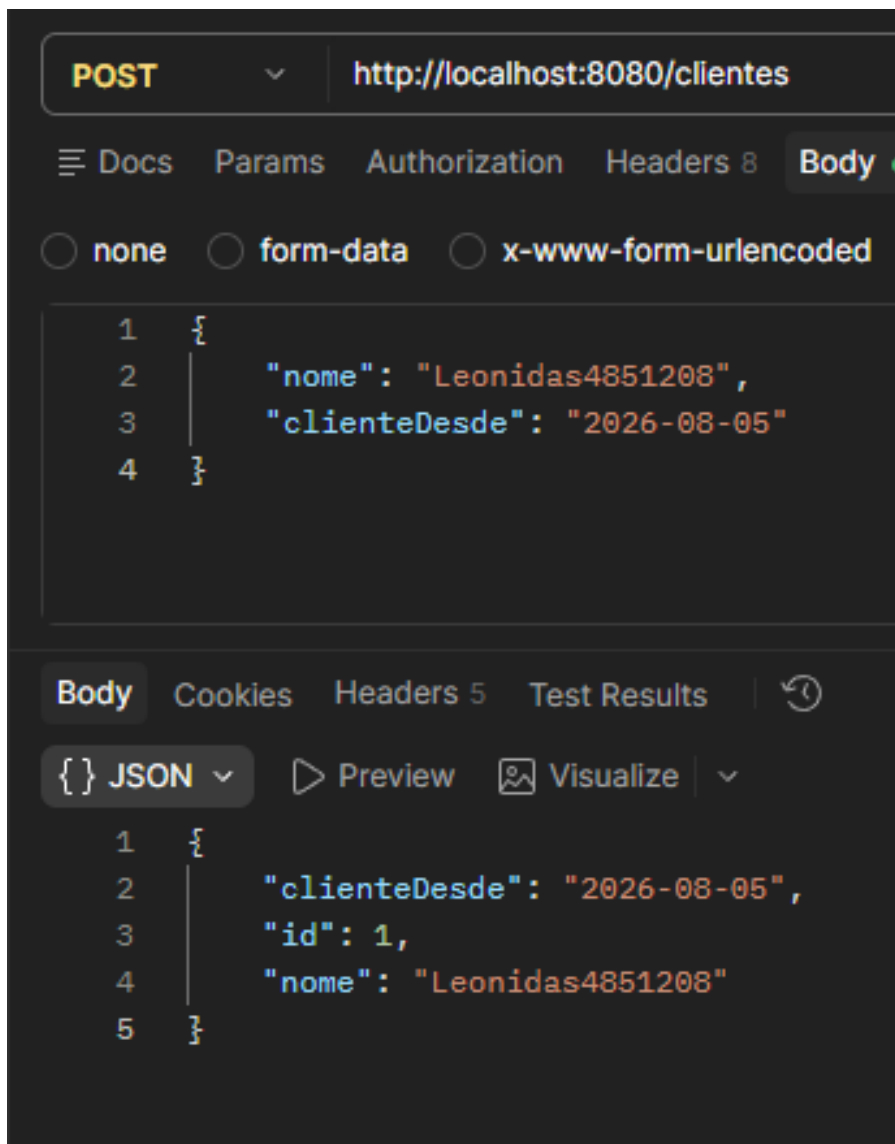
16. Postman Test Evidence

The application was validated using Postman to verify all implemented REST endpoints.

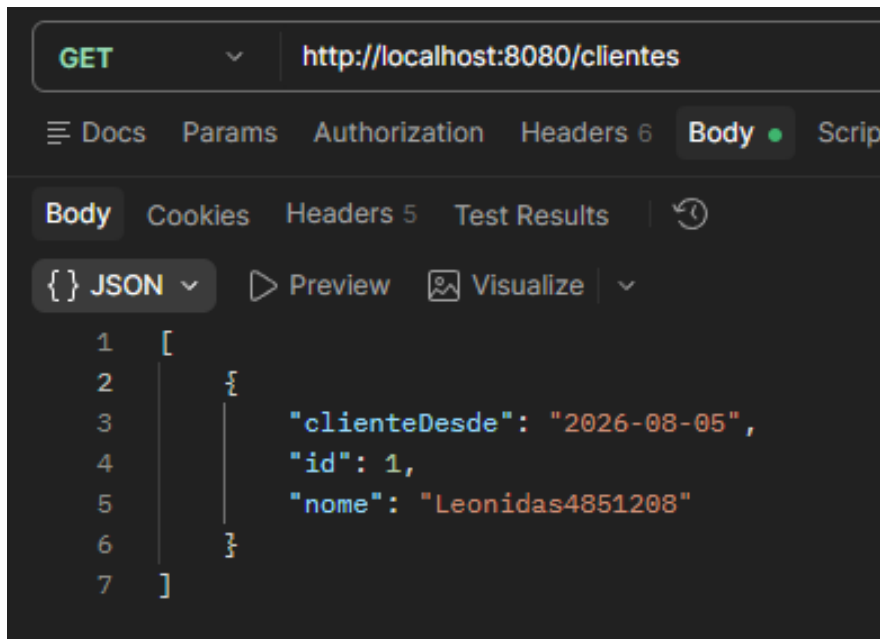
The following operations were successfully tested:

- Customer Creation (POST)
- Customer Listing (GET)
- Customer Search by ID (GET)
- Customer Deletion (DELETE)

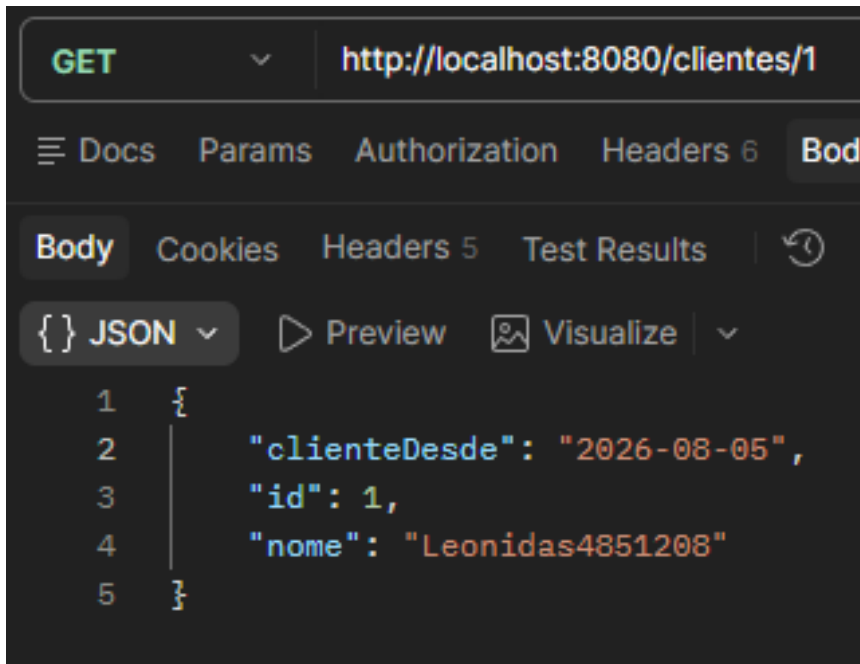
Screenshot – Customer POST



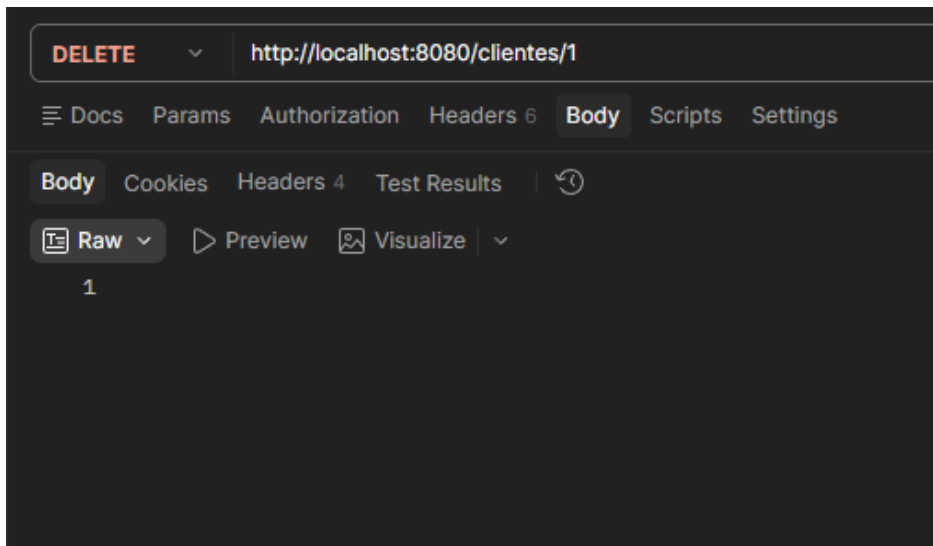
Screenshot – Customer GET



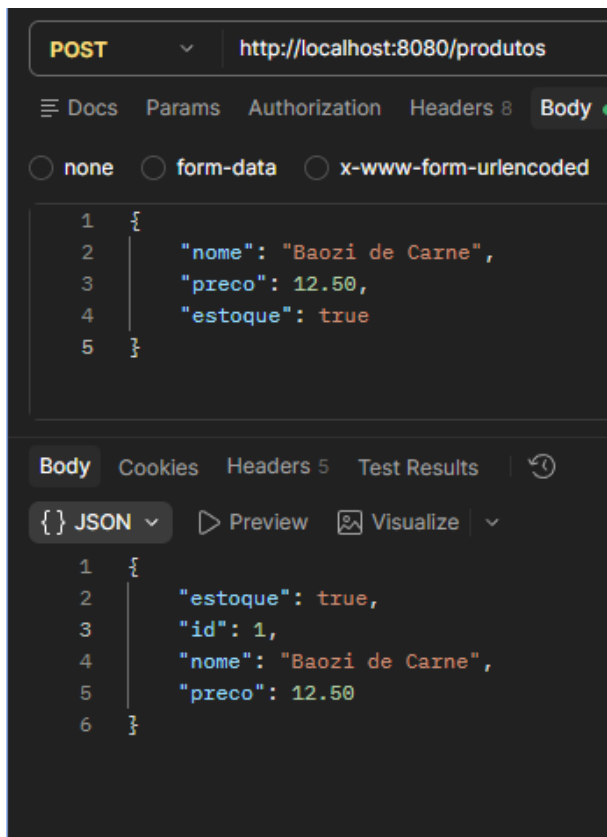
Screenshot – Customer GET by ID



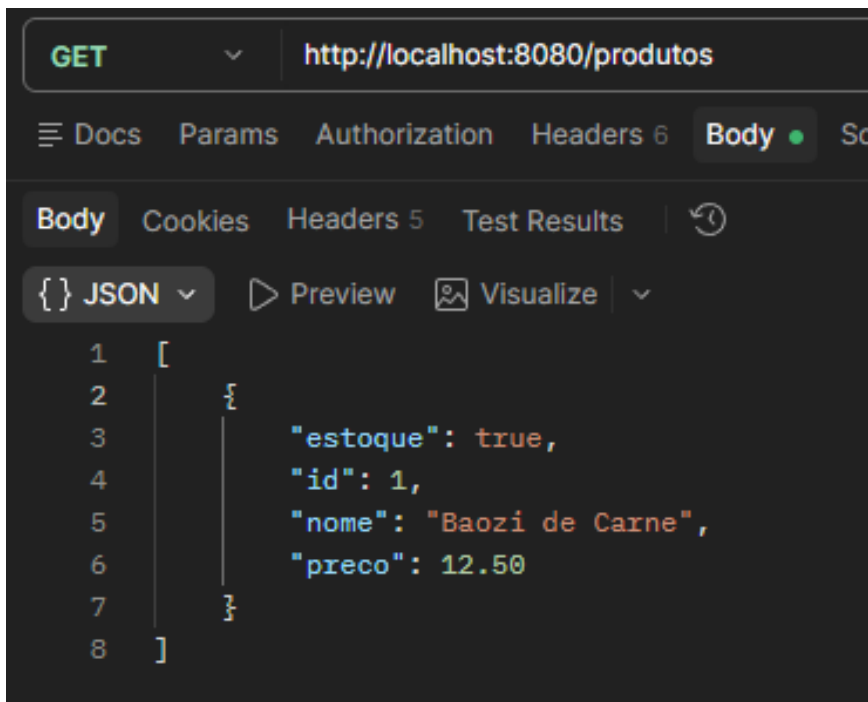
Screenshot – Customer DELETE



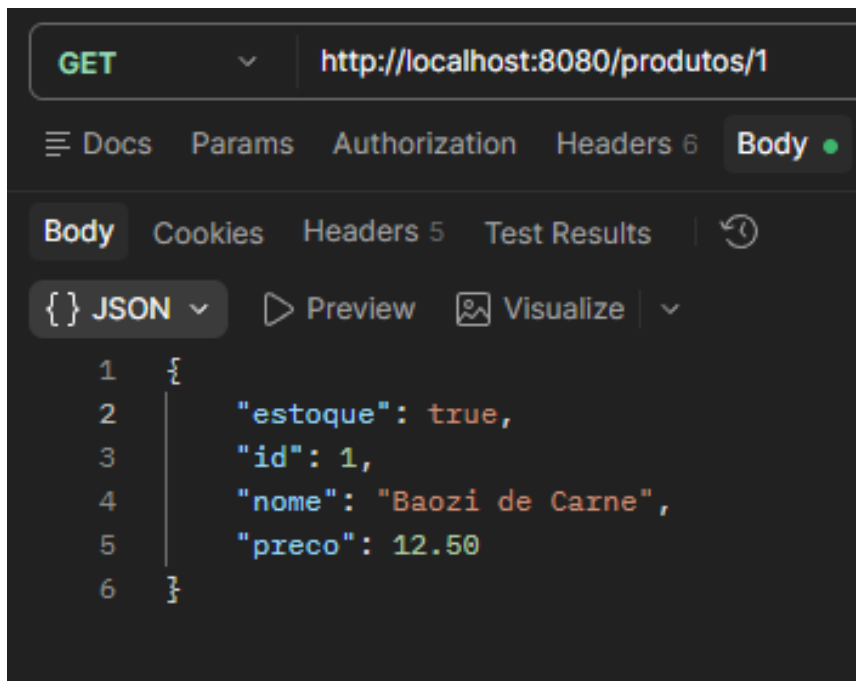
Screenshot – Product POST



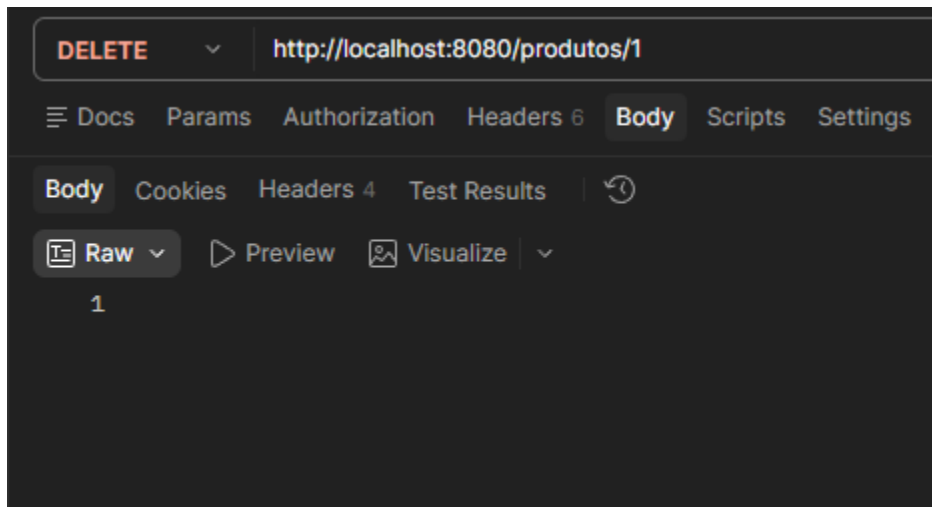
Screenshot – Product GET



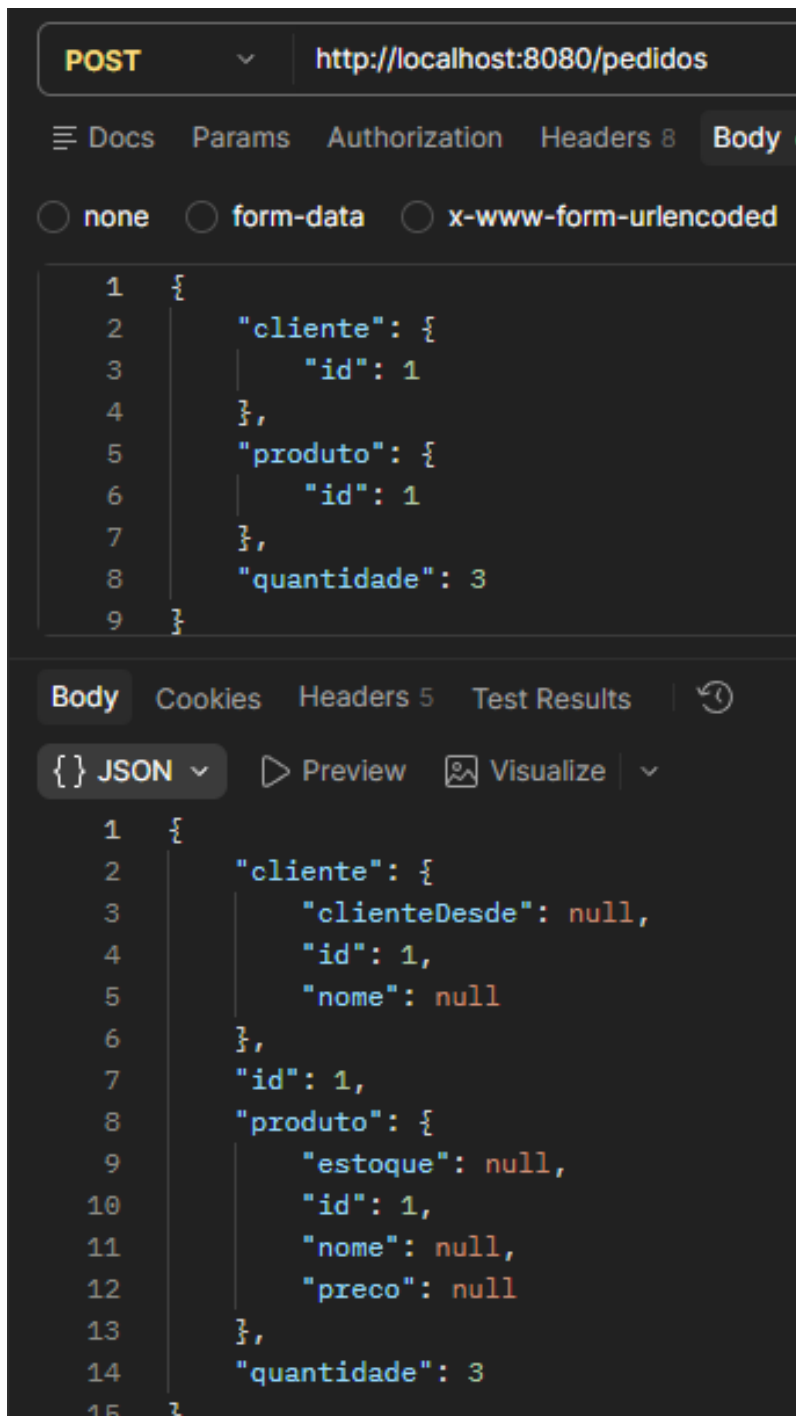
Screenshot – Product GET by ID



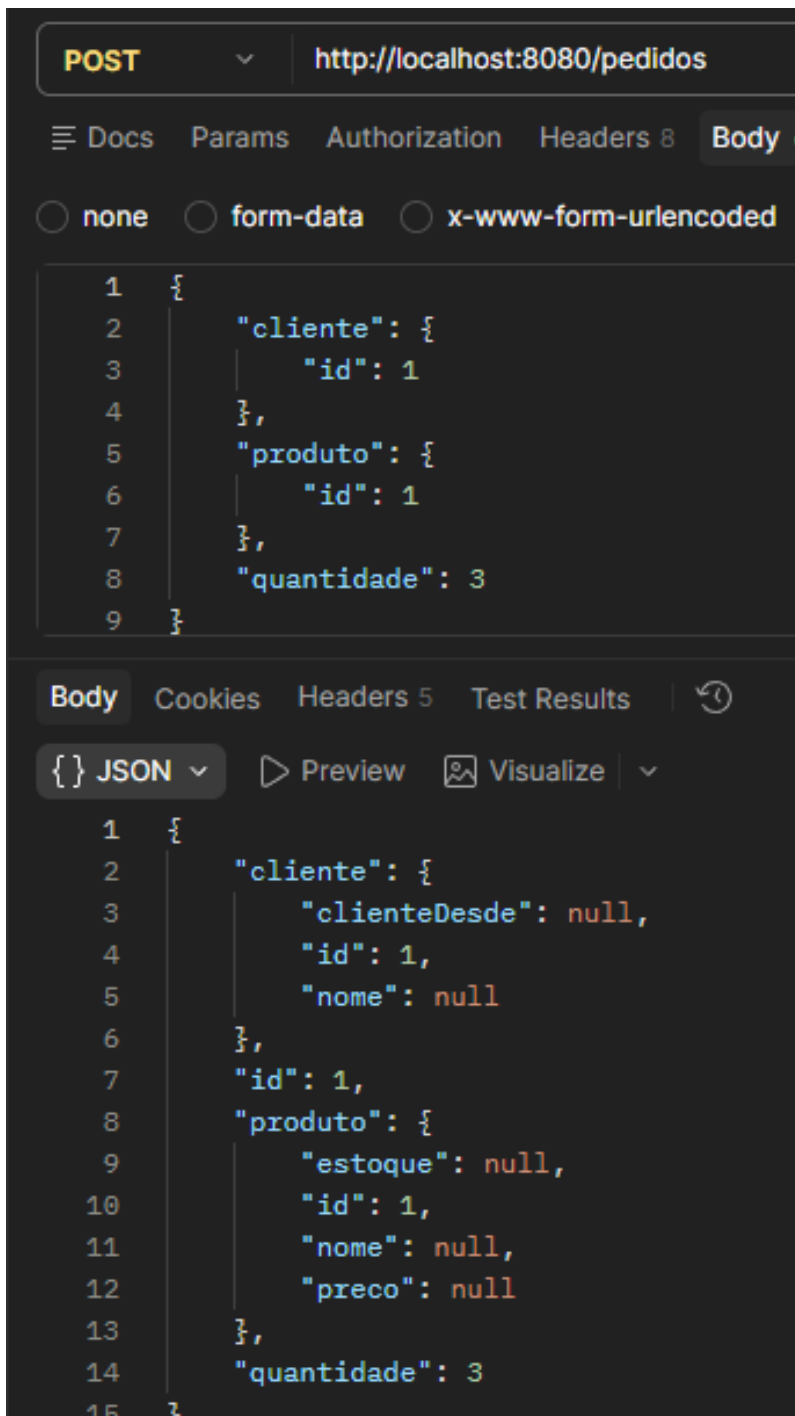
Screenshot – Product DELETE



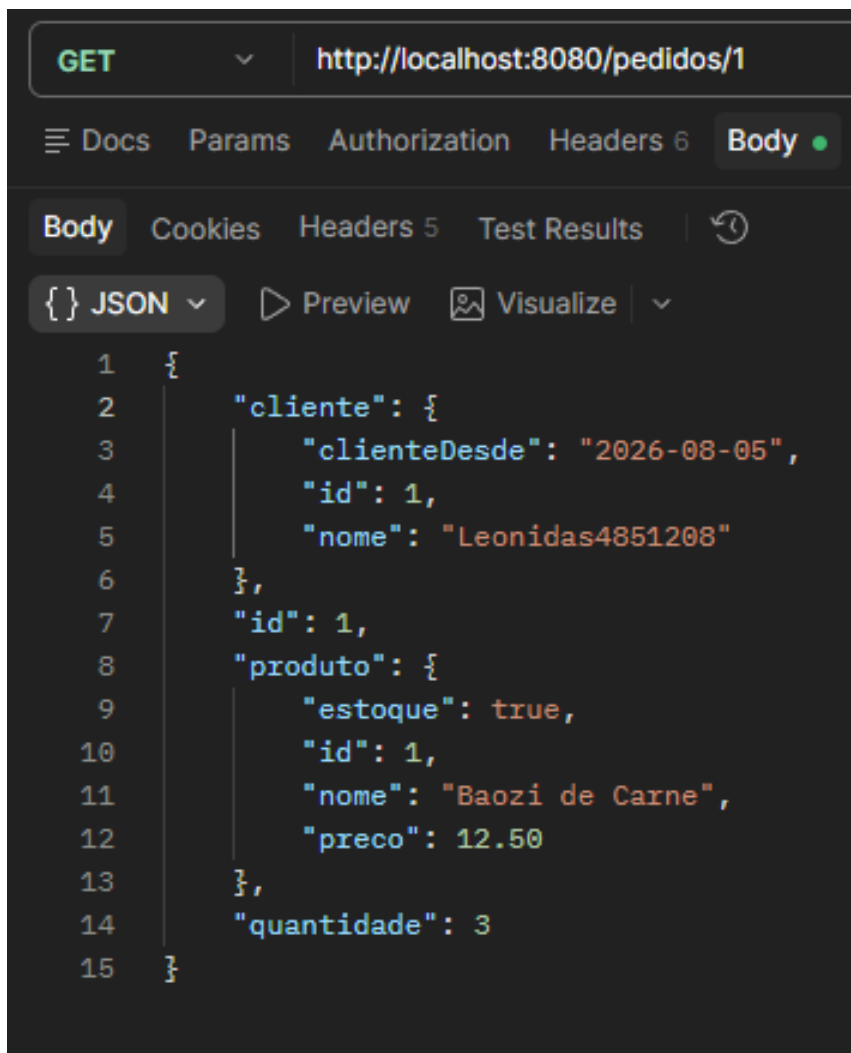
Screenshot – Order POST



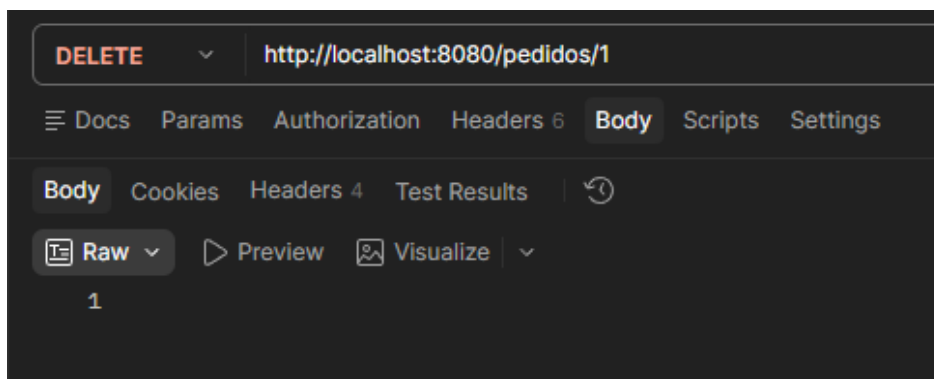
Screenshot – Order GET



Screenshot – Order GET by ID



Screenshot – Order DELETE



17. Installation Guide

17.1 Clone Repository

```
git clone https://github.com/leonndas/store-management-api
```

17.2 Navigate to Project

```
cd store-management-api
```

17.3 Configure Database

Update the **application.properties** file with the MySQL connection settings.

Example:

```
spring.datasource.url=jdbc:mysql://localhost:3306/storedb
```

```
spring.datasource.username=root
```

```
spring.datasource.password=your_password
```

17.4 Run Application

```
mvn spring-boot:run
```

The application will start on:

<http://localhost:8080>

18. Conclusion

The Store Management API successfully demonstrates the implementation of a RESTful backend application using Java and Spring Boot.

Throughout the project, core software engineering concepts such as layered architecture, object-relational mapping, dependency injection and RESTful service design were applied to create a maintainable and extensible solution.

The application provides complete CRUD operations for customers, products and orders while persisting data in a MySQL relational database through Spring Data JPA and Hibernate.

In addition to meeting the functional requirements, the project serves as a practical demonstration of backend development skills, including API design, project organization, database integration and endpoint validation using Postman.

The modular architecture adopted during development allows future enhancements without requiring major structural changes, making the project a solid foundation for more advanced enterprise applications.

19. References

- Oracle. *Java Documentation*.
- Spring. *Spring Boot Documentation*.
- Spring. *Spring Data JPA Reference*.
- Hibernate ORM Documentation.
- MySQL Documentation.
- Apache Maven Documentation.
- Git Documentation.
- GitHub Documentation.
- Postman Learning Center.

Repository

<https://github.com/leonndas/store-management-api>

Project Information

Item	Information
Project Name	Store Management API
Author	Leon Santiago Gomes
Language	Java
Framework	Spring Boot
Database	MySQL
Build Tool	Maven
Version	1.0.0
Repository	GitHub
